



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



GUÍA PRÁCTICA SEMANA 12

Asignatura: Desarrollo de Aplicaciones Web (IS093A)

Unidad II: Desarrollo Web Fullstack

Tema: Gestión de Formularios, Django Admin, Middleware, Sesiones, Autenticación y Autorización

Modalidad: Presencial / Laboratorio

Fecha: 24.06.2026

Duración: 60 minutos

APELLIDOS Y NOMBRES: Cruz Cruz Alexander

OBJETIVO DE LA PRÁCTICA

Dominar el ciclo completo de gestión de datos y seguridad en Django 5.x mediante la implementación de **formularios robustos** (clásicos y [ModelForm](#)), validación en múltiples etapas ([clean\(\)](#), [clean_<campo>\(\)](#), validadores personalizados), sanitización de entrada, integración con plantillas y manejo de errores estructurados. Configurar y personalizar el **Django Admin** para administración operativa (registro, filtros, búsquedas, campos calculados, acciones masivas). Comprender e implementar el **ciclo de Middleware** (interceptación [request/response](#)), gestión de **sesiones seguras**, autenticación ([login/logout/@login_required](#)), autorización basada en roles/permisos, y protección CSRF. Integrar todos los componentes en un sistema backend funcional, auditado y alineado a estándares de seguridad web.

RECURSOS REQUERIDOS

- Python 3.11+ + Django 5.x + [pip](#)
- Visual Studio Code (Extensiones: Python, Django, Pylint, SQLite Viewer, REST Client)
- Terminal integrada + entorno virtual ([venv](#))
- Herramientas: [django-extensions](#) + [shell_plus](#), Postman/Insomnia (pruebas auth), DevTools (Cookies/Storage, Network)
- Documentación oficial: docs.djangoproject.com/en/5.0/topics/forms/, [admin/](#), [middleware/](#), [auth/](#)
- GitHub (control de versiones)



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



PARTE 1: EJERCICIO PASO A PASO

Fase	Actividad Detallada	Tiempo	Concepto Profundo	Restricción/Nota Técnica
1	Formularios & Plantillas: Crear forms.py, implementar ModelForm y formulario clásico. Renderizar en template con {{ form.as_p }}, {{ form.errors }}, csrf_token. Configurar POST route.	10 min	Ciclo de vida del formulario: instanciación → validación → cleaned_data → guardado. CSRF middleware inyecta token automáticamente. Separación lógica (form) de presentación (template).	Prohibido usar request.POST directamente para guardar. Siempre pasar a Form(request.POST). Validar form.is_valid() antes de save().
2	Validación & Sanitización: Implementar clean_<campo>(), clean() cross-field, validadores regex, sanitize_input() básico. Mostrar errores por campo vs. _all_.	10 min	Validación en 3 niveles: campo → formulario → modelo. ValidationError detiene flujo y retorna al template. Sanitización previene XSS/SQLi antes de persistencia. strip(), lower(), escape().	No confiar en validación frontend. clean() se ejecuta solo si validaciones de campo pasan. Usar self.add_error() para errores no fatales.
3	Django Admin: Registrar modelos, personalizar list_display, list_filter, search_fields, readonly_fields, actions. Crear campo calculado (@admin.display).	10 min	Admin es una app Django completa. Usa ModelAdmin para sobrescribir comportamiento. list_display acepta métodos. actions recibe request, queryset. Optimizar queries con select_related().	Nunca exponer campos sensibles (password, is_staff). Usar fields/exclude para controlar visibilidad. readonly_fields no se guarda en DB.



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



4	Middleware & Sesiones: Crear middleware custom (<code>_init_</code> , <code>_call_</code>). Configurar <code>SESSION_ENGINE</code> , <code>SESSION_COOKIE_AGE</code> , <code>CSRF_COOKIE_SECURE</code> . Implementar <code>@login_required</code> , <code>PermissionRequiredMixin</code> .	10 min	Middleware ejecuta en orden LIFO en <code>MIDDLEWARE</code> setting. Intercepta request → view → response. Sesiones usan cookie firmada + DB/cache. <code>@login_required</code> redirige a <code>LOGIN_URL</code> . Permisos: <code>user.has_perm()</code> .	Middleware debe retornar response o <code>None</code> . No modificar <code>request.session</code> sin <code>request.session.modified = True</code> . CSRF token obligatorio en formularios POST.
5	Integración & Auditoría: Probar flujo completo: registro → login → acceso restringido → admin → logout. Verificar cookies, headers <code>X-Frame-Options</code> , <code>Content-Security-Policy</code> . Logs de middleware.	5 min	Ciclo completo: Request → Middleware → URL Resolver → View → Form → Model → DB → Template → Response → Middleware. Seguridad por capas.	Validar python <code>manage.py check --deploy</code> . Nunca <code>DEBUG=True</code> en producción. Rotar <code>SECRET_KEY</code> .

USO CONTROLADO DE HERRAMIENTAS DE IA (Enfoque Estricto)

Herramienta	Uso Permitido	Prompt Ejemplo	Uso Prohibido
ChatGPT/Claude/Copilot	Explicar flujo <code>clean()</code> vs <code>clean_<campo>()</code> , depurar CSRF <code>verification failed</code> , generar	"¿Por qué <code>cleaned_data</code> está vacío <code>transform.is_valid()</code> y cómo capturar <code>ValidationError</code> en la vista?"	"Genera el sistema completo de autenticación, admin y middleware"



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



	estructura ModelAdmin		con Django"
Django shell_plus	Probar queries, inspeccionar request.session, validar formularios interactivament e	python manage.py shell_plus	Modificar db.sqlite3 directament e o saltar form.save()
DevTools (Cookies/Storage)	Inspeccionar sessionid, csrftoken, validar Secure/HttpOnly y flags	(Automático)	Ignorar advertencia s de Mixed Content o SameSite lax

Regla de laboratorio: El 80% de la lógica de validación, middleware y configuración de seguridad debe escribirse manualmente. Cada uso de IA para resolver errores de CSRF, sesiones o validación cruzada debe llevar:

IA: [problema] → Solución manual: [explicación técnica].

Evidencias del Ejercicio Desarrollado:

- Añada capturas de pantalla de cada paso realizado.
- Proporcione el enlace de Github de su proyecto



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



Paso 01: Formularios & Plantillas: Crear forms.py, implementar ModelForm y formulario clásico. Renderizar en template con {{ form.as_p }}, {{ form.errors }}, csrf_token. Configurar POST route.

Qué se crea

- Un archivo forms.py para definir formularios reutilizables.
- Un formulario clásico con forms.Form para registro simple.
- Un formulario basado en modelo con forms.ModelForm cuando se quiere editar o crear instancias de un modelo.

```
from django import forms

class RegistroForm(forms.Form):
    username = forms.CharField(
        max_length=100,
        widget=forms.TextInput(attrs={
            'placeholder': 'Tu usuario',
        }),
    )
    email = forms.EmailField(
        widget=forms.EmailInput(attrs={
            'placeholder': 'correo@dominio.com',
        }),
    )
    password = forms.CharField(
        widget=forms.PasswordInput(attrs={
            'placeholder': 'Crea una contraseña',
        }),
    )
    confirm_password = forms.CharField(
        widget=forms.PasswordInput(attrs={
            'placeholder': 'Repite la contraseña',
        }),
    )

    def clean_username(self):
        username = self.cleaned_data['username']

        if len(username) < 4:
            raise forms.ValidationError("Muy corto")

        return username

    def clean(self):
        cleaned_data = super().clean()

        password = cleaned_data.get('password')
        confirm = cleaned_data.get('confirm_password')

        if password and confirm and password != confirm:
            raise forms.ValidationError("Contraseñas distintas")

        return cleaned_data

from .models import Perfil

class PerfilForm(forms.ModelForm):
    class Meta:
        model = Perfil
        fields = ['telefono', 'edad']
        widgets = {
            'telefono': forms.TextInput(attrs={'placeholder': 'Ej. 987654321'}),
            'edad': forms.NumberInput(attrs={'min': '1', 'placeholder': 'Edad'}),
        }
```



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



Paso 02: Validación & Sanitización: Implementar `clean_()`, `clean()` cross-field, validadores regex, `sanitize_input()` básico. Mostrar errores por campo vs. `__all__`.

Una vez que el formulario existe, se agregan reglas para evitar datos inválidos o inseguros.

Validaciones por campo

Se usan métodos como `clean_<campo>()` para validar un campo específico.

- Sirve para revisar longitud mínima, formato o restricciones propias de un dato.
- Si falla, el error se asocia a ese campo concreto.

Validación cruzada

Se usa `clean()` para revisar más de un campo al mismo tiempo.

- Por ejemplo, comprobar que `password` y `confirm_password` sean iguales.
- Si el problema afecta a varios campos o al formulario completo, el error puede ir a `__all__`.

Validadores y regex

Se pueden usar validadores personalizados o expresiones regulares para:

- Aceptar solo caracteres permitidos.
- Bloquear datos con formato incorrecto.
- Evitar entradas no deseadas en campos sensibles.

Sanitización básica

La sanitización consiste en limpiar los datos antes de usarlos.

- Eliminar espacios al inicio o al final.
- Normalizar cadenas antes de guardarlas.
- Rechazar valores vacíos o incompletos cuando sea necesario.

```
import html

def sanitize_input(value):
    return html.escape(value.strip().lower())

def clean_username(self):
    username = sanitize_input(self.cleaned_data['username'])
    return username
```

Paso 03: Django Admin: Registrar modelos, personalizar `list_display`, `list_filter`, `search_fields`, `readonly_fields`, `actions`. Crear campo calculado (`@admin.display`).

Qué se hace

- Registrar modelos en `admin.py`.
- Personalizar la lista de objetos con `list_display`.
- Filtrar resultados con `list_filter`.
- Buscar con `search_fields`.
- Proteger campos con `readonly_fields`.
- Agregar acciones en bloque con `methods` de `admin`.

Campo calculado

Se puede crear un valor derivado con `@admin.display`.

- No viene de la base de datos directamente.
- Se calcula al mostrarlo en el admin.
- Es útil para estados, etiquetas o clasificaciones.



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



```
from django.contrib import admin
from .models import Perfil

@admin.register(Perfil)
class PerfilAdmin(admin.ModelAdmin):

    list_display = ('user', 'telefono', 'edad', 'estado')
    list_filter = ('edad',)
    search_fields = ('user__username',)
    readonly_fields = ('estado',)

    @admin.display(description="Estado")
    def estado(self, obj):
        return "Mayor" if obj.edad >= 18 else "Menor"
```

Paso 04: Django Admin: Registrar modelos, personalizar list_display, list_filter, search_fields, readonly_fields, actions. Crear campo calculado (@admin.display).

```
from django.contrib import admin
from .models import Perfil

@admin.register(Perfil)
class PerfilAdmin(admin.ModelAdmin):

    list_display = ('user', 'telefono', 'edad', 'estado')
    list_filter = ('edad',)
    search_fields = ('user__username',)
    readonly_fields = ('estado',)

    @admin.display(description="Estado")
    def estado(self, obj):
        return "Mayor" if obj.edad >= 18 else "Menor"
```



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



Paso 05: Integración & Auditoría: Probar flujo completo: registro → login → acceso restringido → admin → logout. Verificar cookies, headers XFrame-Options, ContentSecurity-Policy. Logs de middleware.

```
class LogMiddleware:

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        print("Request:", request.path)

        response = self.get_response(request)

        print("Response:", response.status_code)

        return response
```

```
PS C:\Users\alexa\Downloads\web\sisitema> python manage.py check --deploy
?: (security.W012) SESSION_COOKIE_SECURE is not set to True. Using a secure-only session cookie makes it more difficult for network traffic sniffers to hijack user sessions.
?: (security.W016) You have 'django.middleware.csrf.CsrfViewMiddleware' in your MIDDLEWARE, but you have not set CSRF_COOKIE_SECURE to True. Using a secure-only CSRF cookie makes it more difficult for network traffic sniffers to steal the CSRF token.
?: (security.W018) You should not have DEBUG set to True in deployment.
?: (security.W020) ALLOWED_HOSTS must not be empty in deployment.

System check identified 7 issues (0 silenced).
PS C:\Users\alexa\Downloads\web\sisitema>
```

```
PS C:\Users\alexa\Downloads\web\sisitema> python manage.py runserver
Response: 200
[24/Jun/2026 13:52:32] "GET /login/ HTTP/1.1" 200 1978
[24/Jun/2026 13:52:32] "GET /static/usuarios/styles.css HTTP/1.1" 304 0
Request: /login/
Response: 302
[24/Jun/2026 13:52:42] "POST /login/ HTTP/1.1" 302 0
Request: /dashboard/
Response: 200
[24/Jun/2026 13:52:42] "GET /dashboard/ HTTP/1.1" 200 2186
```




UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



127.0.0.1:8000/login/

SP

SISTEMA PRIVADO

Registro Acceso Panel

BIENVENIDO DE VUELTA

Accede a tu panel

Usa tus credenciales para entrar al sistema y continuar con tu gestión.

Usuario

Tu usuario

Contraseña

Tu contraseña

Entrar

¿No tienes cuenta? [Crea una aquí](#)

127.0.0.1:8000/dashboard/

SP

SISTEMA PRIVADO

Registro Acceso Panel

ÁREA PRIVADA

Cerrar sesión

Dashboard

Sesión iniciada correctamente. Desde aquí puedes seguir creciendo el sistema sin perder el flujo visual.

ACCESO

Seguro

Autenticación y sesión activas con una experiencia más cuidada.

INTERFAZ

Premium

Tarjetas, sombras suaves y jerarquía visual más marcada.

BASE

Lista

El proyecto ya tiene una estructura visual consistente para seguir creciendo.

SIGUIENTE PASO

Podemos extender este estilo al resto de páginas del sistema.

Si quieres, el siguiente ajuste puede incluir una barra superior real, iconografía, tablas y un dashboard con métricas dinámicas.

Repositorio:

<https://github.com/alexandercruzjh/semana12>